



University of
Nottingham

UK | CHINA | MALAYSIA

COMP2001/2011: Artificial Intelligence Methods

Lecture 2 – Components of Heuristic Search

Prof Ender Ozcan



Last Week

- Introduction of the terminologies.
- Motivation for in-exact/heuristic search.
- Bin Packing and Travelling Salesman problems.
- Basic constructive and perturbative operators.



Today's Topics

- Introduction to more COPs.
- Main components of (meta/hyper-) heuristic search methods.
- Focus on (hill-climbing/local search) methods.



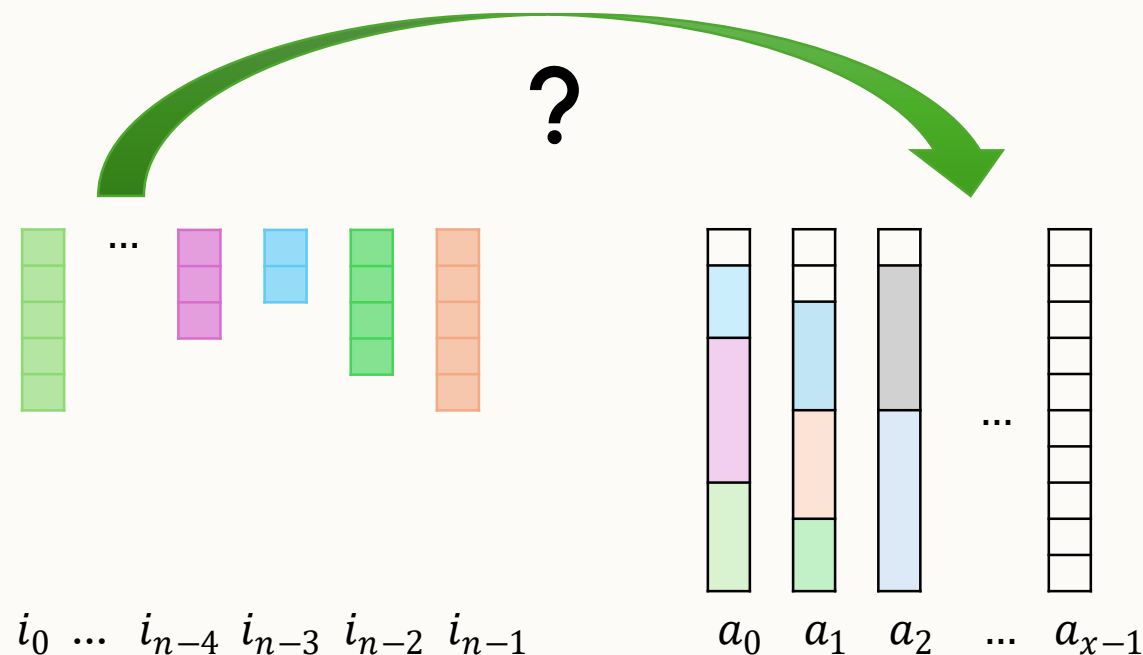
Introduction to more Combinatorial Optimisation Problems

- Bin Packing Problem [Recap]
- Travelling Salesman Problem [Recap]
- 0/1 Knapsack Problem
- Boolean Satisfiability Problem



Bin Packing Problem (Recap)

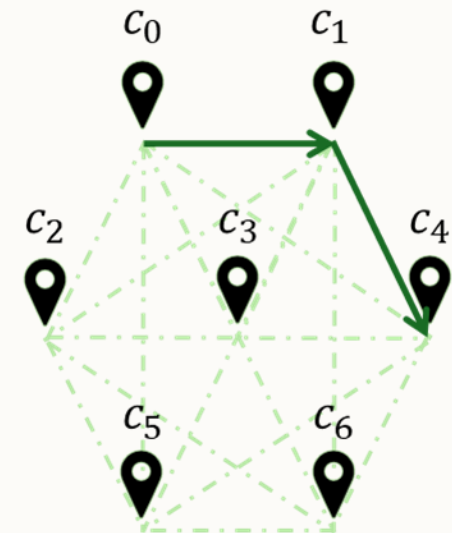
- Given a set of items I , $|I| = n$, each with a size of i_j , where $0 \leq j \leq n - 1$ and an unlimited number of bins each with fixed size/volume V ...
- ...what is the minimum number of bins (that is x) required to pack all items without exceeding the capacity of any of the bins?





Travelling Salesman Problem (Recap)

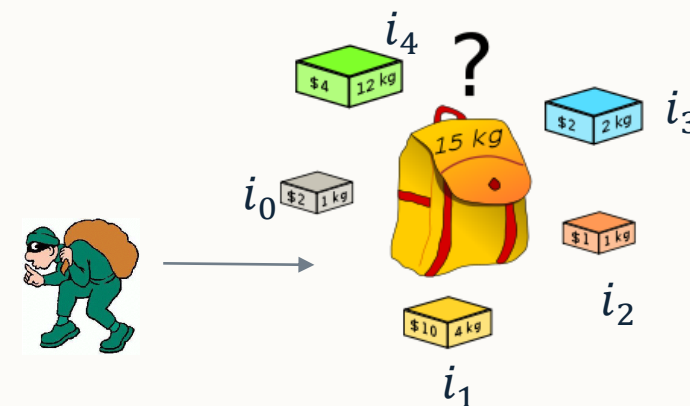
- Given a set of cities C of size N , and distances between each pair of cities $d(i, j)$, for all $i \in C, j \in C \dots$
- ...what is the shortest possible route that visits each city exactly once and returns to the origin city?





0/1 Knapsack Problem

- Given a knapsack with capacity C , and a set of items I , $n = |I|$, where each item has a weight w_x and value v_x , $0 \leq x \leq n - 1 \dots$
- ...which subset items should be packed into the knapsack to maximise the total value?





Boolean Satisfiability Problem

- First problem to be proven to be NP-Complete.
- Given a formula φ of n Boolean variables x_i where $0 \leq i < n$, in conjunctive normal form (CNF)...
- Is there an assignment of True/False values to each of the variables such that φ is satisfied?
- Example formula in CNF:
 - $(\neg x_0 \vee \neg x_1) \wedge (x_1 \vee \neg x_2) \wedge (x_0) \wedge (x_2)$

Note the objective – this is in a problem which we call a decision problem!



Decision Problems vs COPs

- Decision Problems:

- Seek the existence of a satisfaction to a problem.
- Output is a binary decision.
- Depending on the problem, can be simpler to solve than a COP.

- COPs

- Seek the best solution to a problem.
- Output is an objective value or complete solution
- Usually more complex as we are looking for the optimal solution among a set of possibilities.

Example: Consider a SAT problem which contains (among others) the clause $(x_1 \wedge \neg x_1) \dots$



Maximum Satisfiability Problem

- Given a formula φ of n Boolean variables x_i where $0 \leq i < n$, in conjunctive normal form (CNF)...
- ... what is the greatest number of clauses that can be satisfied in φ ?
- Example:
 - $\varphi = (\neg x_0 \vee \neg x_1) \wedge (x_1 \vee \neg x_2) \wedge (x_0) \wedge (x_2)$
 - $X = \langle x_0: True, x_1: False, x_2: False \rangle$
 - $f(X) = 1 + 1 + 1 + 0$
 - $f(X) = 3$
- Note on minimisation variant.



Main Components of Heuristic Search Methods

Overview – what are we going to look at today?



Main Components of (Meta/Hyper-) Heuristic Search Methods

- Search process (guidelines)
- Solution representation (encoding)
- Neighbourhood relations (move operators/low-level heuristics)
- Evaluation function (objective function)
- Initialisation method*
- Mechanism(s) for escaping local optima*

* required for metaheuristics and hyper-heuristics.



Main Components of Heuristic Search Methods

Representations



Representation

- A **representation** is the way to encode a solution to a given problem.
- Specific to each problem.



Characteristics of Representations

- **Completeness** – a solution encoding is complete if it is able to represent all possible solutions to the problem.
- **Connexity** – a solution encoding is connex if for all candidate solutions there exists a search path between itself and all other possible solutions in the search space.
- **Efficiency** – a solution encoding is efficient if it is fast and easy to manipulate it



Binary (Bitstring) Representation

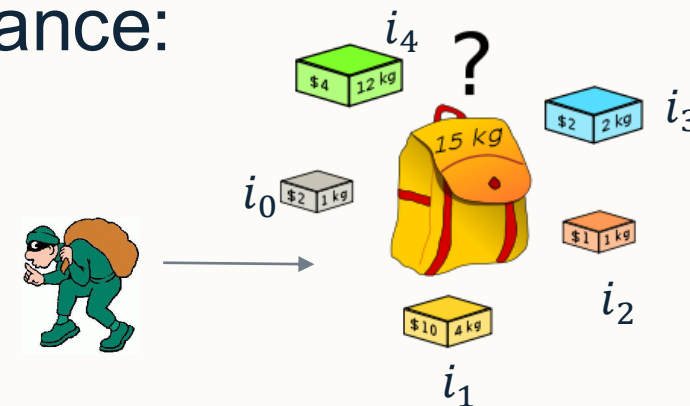
- [illegible]



Binary Representation Example

- Given the following 0/1 Knapsack Problem instance:

- $C = 15$
- $n = 5$
- $W = \{1.0, 4.0, 1.0, 2.0, 12.0\}$
- $V = \{2.00, 10.00, 1.00, 2.00, 4.00\}$



- We can use the binary representation to encode the following solutions:
 - $\langle 10011 \rangle : V = 8, W = 15$
 - $\langle 11110 \rangle : V = 15, W = 8$



Permutation Representation

- Typically used when we are searching for a sequence of N events.
- Each entity in the permutation has an index which represents the order.
- Search space is $N!$

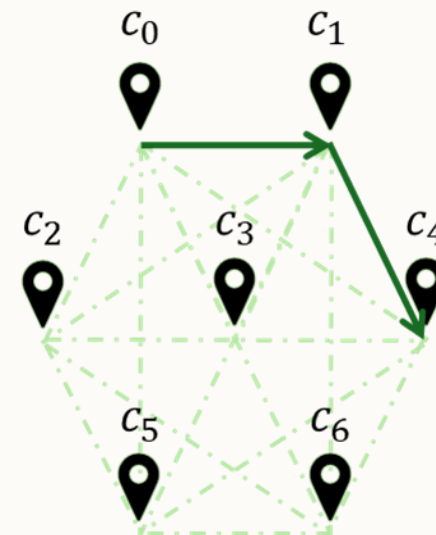


Permutation Representation Example

- Also called **path representation** in TSP.
- Given the following TSP instance:

■ $N = 5$ and $d(i, j) =$

$i \backslash j$	c_0	c_1	c_2	c_3	c_4
c_0	0	5	3	7	9
c_1	5	0	2	12	13
c_2	3	2	0	15	18
c_3	7	12	15	0	21
c_4	9	13	18	21	0



- We can use this representation to encode the following solutions:
 - $s_1 : \langle c_0, c_1, c_2, c_3, c_4 \rangle \quad f(s_1) = 5 + 2 + 15 + 21 + 9 = 52$
 - $s_2 : \langle c_0, c_2, c_1, c_3, c_4 \rangle \quad f(s_2) = 3 + 2 + 12 + 21 + 9 = 47$
- Which heuristic could perturb s_1 to s_2 ? [Answer: pairwise exchange (from Lecture 1)]



Integer Encoding

- Can be used to denote choices within each entity references by its index in the representation.
- E.g.: Personnel rostering problem, timetabling problem, layout/**structure optimisation**
 - Given M types of materials, each integer could represent a type of composite material used in an N -layer composite structure.
 - Search space is M^N where N is the length of the representation (number of layers) and M is the number of choices (types of materials).



Integer Encoding Example

- Given a set of composite materials C , $|C| = M = 5$
- ...and a structure containing 8-layers.

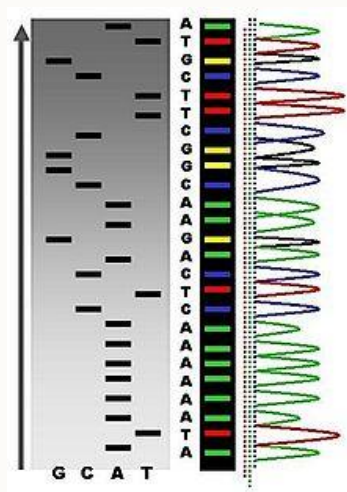
Search space size would be 5^8

- We can use integer encoding to represent the following solutions:
 - $\langle 11235532 \rangle$
 - $\langle 52312315 \rangle$



Value Encoding

- Generally used for continuous optimisation with real values.
- But can be used for others such as parameter optimisation (e.g., 0.2324 0.3243 0.4556 0.3293 0.9545), DNA sequencing (e.g., ATGCTTCGGCAAGACTCAAAAAATA) and planning – here looking for orders of instructions.
- Search space is context dependent.

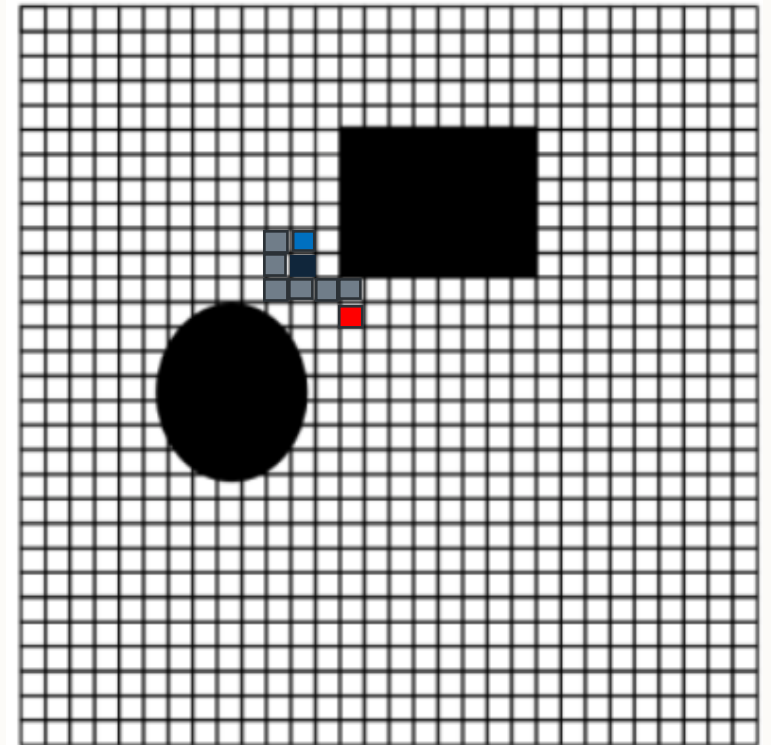




Value Encoding Example

- Given a starting point, an end point, and a set of items to sample
- ...find a path from the start to the end which samples all items.

- We can use value encoding to form a plan:
< (Up), (Left), (Left), (Left), (Up), (Up) (Right) >





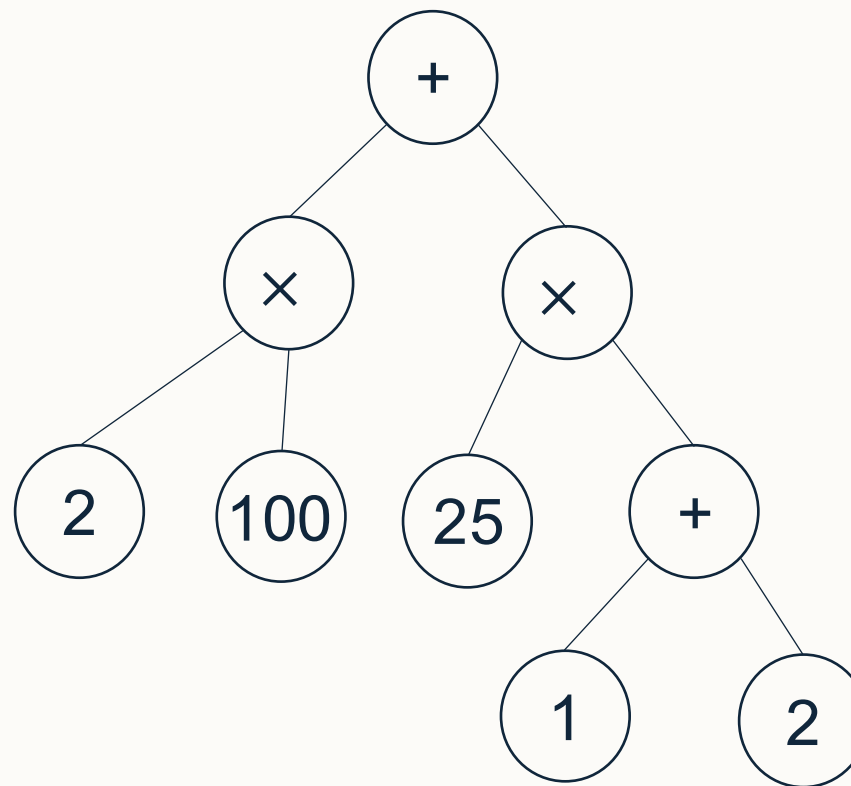
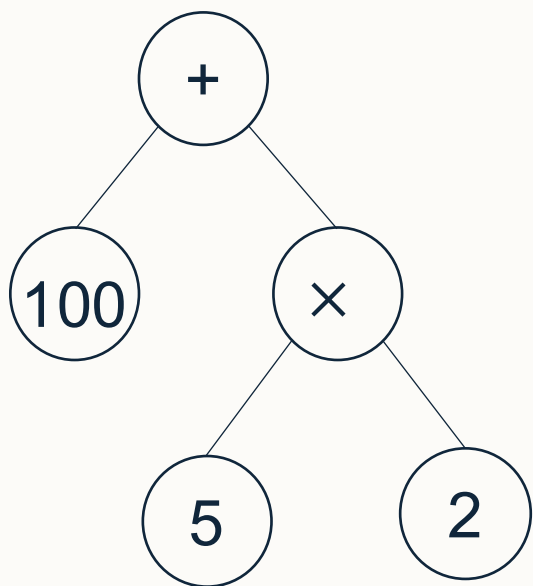
Non-linear Encodings

- Such as **tree encoding** which can be used to generate programmes.
- Used in genetic programming (we will cover this in another lecture) where we can generate heuristics and components of heuristics.
- Search space context dependent.



Non-linear Encodings Example

- Given the list of numbers $\{1, 1, 2, 5, 25, 100\}$ find a way to calculate 275 using the operators $(+, -, \times, /)$.





Exercise – Representations

- **Exercise** - Which of the representations discussed would be the most suitable for encoding a maximum satisfiability problem?
 - A. Binary
 - B. Permutation
 - C. Integer
 - D. Value
 - E. Non-linear



MAX-SAT Problem Example

- Given a formula φ of n Boolean variables x_i where $0 \leq i < n$, in conjunctive normal form (CNF)...
- ... what is the greatest number of clauses that can be satisfied in φ ?
- Given $\varphi = (x_0 \vee x_1) \wedge (\neg x_0 \vee x_1) \wedge (x_0 \vee \neg x_1) \wedge (\neg x_0 \vee \neg x_1)$
- How many variables?
 - 2
- Hence need a binary string of length 2 to encode a solution to this problem instance.
- What is the search space size?
 - 2^n , hence there are 4 possible solutions.



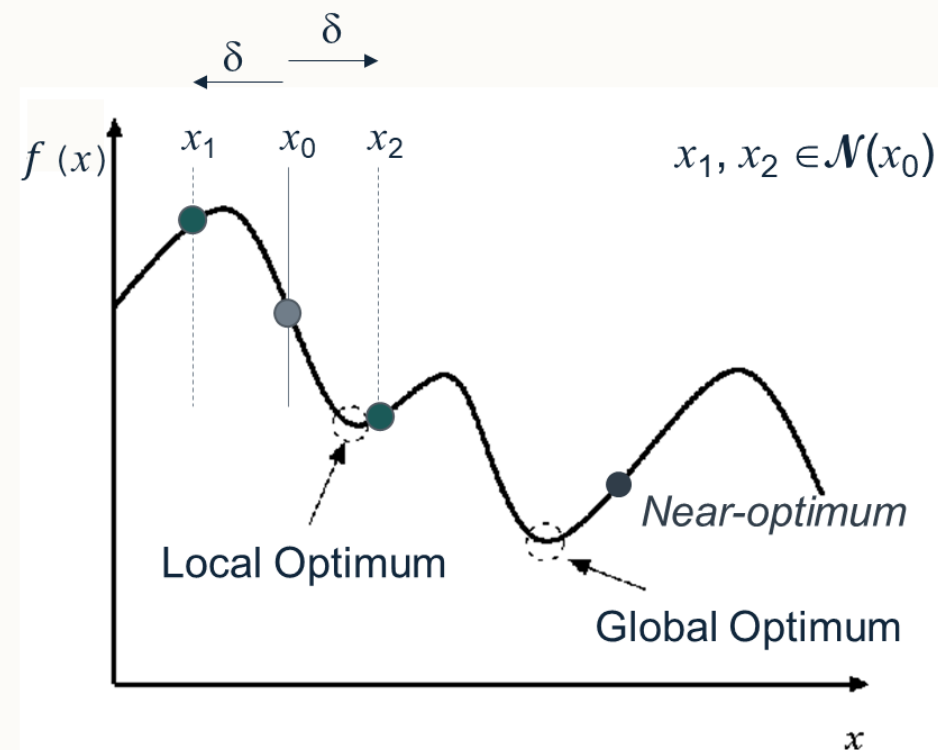
Main Components of Heuristic Search Methods

Neighbourhoods - How can we move between solutions?



Definition – Neighbourhoods

- A **neighbourhood** of a solution is a set of solutions that can be reached from that solution by applying an (operator/move operator/heuristic).
- Take x_0 to be the current solution with the operator being $\pm\delta$.
- The neighbourhood of x_0 is the set of solutions $\{s_1, s_2\}$.





Example Neighbourhood for Binary Represented Solutions

- A **bit-flip operator** flips a bit in a given solution.
 - Example: 00110 $\mapsto$ 00111
- The **hamming distance** between two bitstrings of equal length is the number of positions at which the respective symbols differ.
 - Example: $\text{HD}(00110, 00111) = 1$.
- For a binary string of length n , the neighbourhood size is n and hamming distance is 1.



Example Neighbourhood for Integer and Value Represented Solutions

- A random (perturbation/move/neighbourhood/assignment) operator replaces a discrete value with another from a given alphabet Σ .
 - Example: given $\Sigma = \{0,1,2,3\}$ and $X = 0023 \mapsto 2023$.
- For a solution of size n and alphabet of size k , the neighbourhood size is: ???
 - $(k - 1)n$
 - Example: $3 \times 4 = 12$.
- Hamming distance of a single random perturbation is 1.



Example Neighbourhoods for Permutation Represented Solutions

- Many operators to define different neighbourhoods:
 - Adjacent (pairwise interchange/swap).
 - Insertion operator.
 - Exchange operator.
 - Inversion operator.



Adjacent Pairwise Interchange for Permutation Represented Solutions

- The **adjacent (pairwise swap/exchange/interchange) operator** swaps two adjacent entries in the permutation.
 - Example: **5**1432 $\mapsto$ **1**5432
 - For a permutation of size n , the neighbourhood size is $n - 1$.
 - Hamming distance is always 2.
-
- Note: sometimes the boundary case is considered (can swap the 2 and 5) leading to a neighbourhood size of n .



Insertion for Permutation Represented Solutions

- The **insertion operator** removes an entry from the permutation and inserts it into a different position.
- Example: **5**1432 $\mapsto$ 143**5**2
- For a permutation of size n , the neighbourhood size is $(n - 1)^2$
- Hamming distance is dependent on the number of places an item is shifted, $2 \leq HD \leq n$
- Note it is also possible that **5**1432 $\mapsto$ **1**5432 etc. and hence subsumes adjacent swap.



Exchange for Permutation Represented Solutions

- The **(pairwise) exchange operator** swaps two arbitrarily, but different, selected entries.
 - Example: $51432 \mapsto 31452$
 - For a permutation of size n , the neighbourhood size is $n(n - 1) / 2$.
 - Hamming distance is always 2.
-
- Note it is also possible that $51432 \mapsto 15432$ etc. and hence subsumes adjacent swap.



Inversion for Permutation Represented Solutions

- The **inversion operator** reverses (inverts) the entries between two arbitrarily selected start and end points.
- Example: $5\mathbf{134}2 \mapsto 5\mathbf{431}2$
- For a permutation of size n , the neighbourhood size is $n(n - 1) / 2$.
- Hamming distance is dependent on the selected start and end points, $2 \leq HD \leq n$.
- Note it is also possible that $\mathbf{51}432 \mapsto \mathbf{15}432$ etc. and hence subsumes adjacent swap.



Main Components of Heuristic Search Methods

Evaluation/Objective Functions



Definition – Evaluation Function

- An **evaluation function** indicates how good a given solution is, distinguishing it between one that is objectively better or worse.
- Also referred to as an (objective/cost/fitness/penalty) function.



Evaluation Functions

- Extremely important for the search process providing feedback on the quality of the solution.
 - Need to select a solution from the neighbourhood – which one?
 - Can be used to guide the search to promising regions.
- Different types:
 - Separable vs non-separable (terms).
 - Unimodal vs multi-modal (number of global optima).
 - Single-objective and multi-objective.
- Could be computationally expensive themselves (e.g. simulation).
- Exact models vs surrogate models – approximate trade-offs.



Evaluation function for MAX-SAT

- Given the total number of clauses n and the total number of satisfied clauses C_{SAT} , we can design many evaluation functions:
 - $f_1 = C_{SAT}$
 - $f_2 = n - C_{SAT}$
 - $f_3 = (n - C_{SAT}) / n$
- These can be designed for maximisation or minimisation.
- Example:
 - $\varphi = (\neg x_0 \vee x_1) \wedge (\neg x_0 \vee \neg x_1)$
 - $s_1 = \langle 1, 1 \rangle$ $f_1(s_1) = 1$; $f_2(s_1) = 1$; $f_3(s_1) = 0.5$
 - $s_2 = \langle 0, 1 \rangle$ $f_1(s_2) = 2$; $f_2(s_2) = 0$; $f_3(s_2) = 0.0$



Evaluation function for MAX-SAT

- Given the total number of clauses n and the total number of satisfied clauses C_{SAT} , we can design many evaluation functions:
 - $f_1 = C_{SAT}$
 - $f_2 = n - C_{SAT}$
 - $f_3 = (n - C_{SAT}) / n$
- These can be designed for maximisation or minimisation.
- Example:
 - $\varphi = (\neg x_0 \vee x_1) \wedge (\neg x_0 \vee \neg x_1)$
 - $s_1 = \langle 1, 1 \rangle$ $f_1(s_1) = 1$; $f_2(s_1) = 1$; $f_3(s_1) = 0.5$
 - $s_2 = \langle 0, 1 \rangle$ $f_1(s_2) = 2$; $f_2(s_2) = 0$; $f_3(s_2) = 0.0$



Evaluation function for TSP

- Given a permutation of $\{0, \dots, N - 1\}$ denoted π and a cost matrix C of dimension $N \times N$ where $c_{i,j}$ denotes a known distance between cities i and j , and $\pi(i)$ denotes the city at the i -th location in the tour:
- Evaluation function for minimising the total path length:

$$f(\pi, C) = \sum_{i=0}^{N-1} c_{\pi(i), \pi((i+1) \bmod N)}$$

- $c_{i,j}$ can be computed as $c_{i,j} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$
- Any problems with calculating this each time? [Answer: computational expense for calculating square and square root]



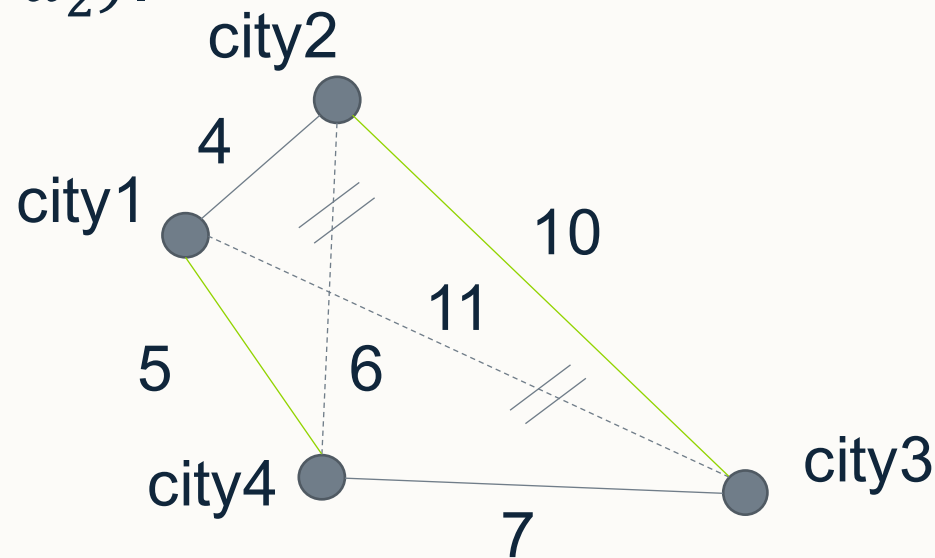
(Delta/Incremental) Evaluation

- **Key idea** is to only update what has been changed.
- **Delta evaluation** is a technique to improve computational efficiency by calculating the effects or differences between the current search position, s , and a neighbouring solution, s' , on the evaluation function value.
- Delta evaluation is crucial for an efficient implementation of heuristics/metaheuristics/hyper-heuristics.



Example – Delta Evaluation for TSP

- Given $s = \langle c_2, c_1, c_3, c_4 \rangle$ with $f(s) = 28$, and we use the pairwise interchange operator to get $s' = \langle c_1, c_2, c_3, c_4 \rangle$
- We can calculate $f(s')$ using delta evaluation as $f(s') = \Sigma + \Delta$ where Σ is $f(s)$ and $\Delta = (n_1 + n_2) - (d_1 + d_2)$.
- $$\begin{aligned} f(s') &= 28 + (5 + 10) - (11 + 6) \\ &= 28 - 2 \\ &= 26 \end{aligned}$$





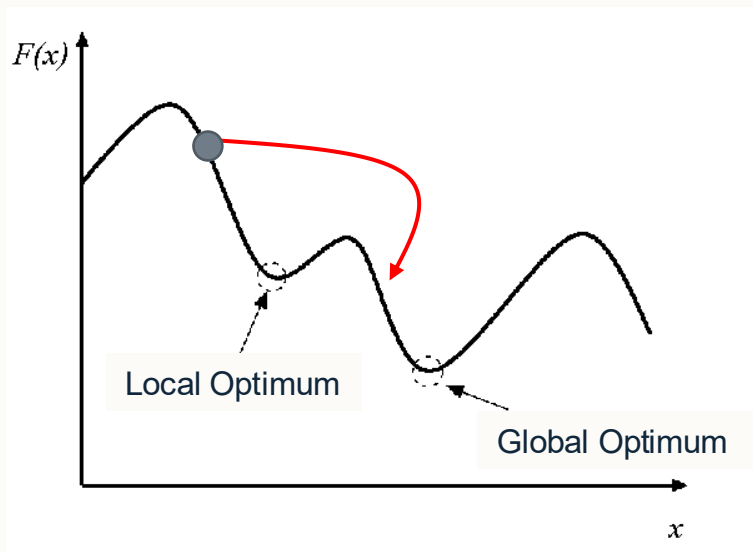
Main Components of Heuristic Search Methods

Hill Climbing Algorithms



Definition – Hill Climbing

- A **hill climbing algorithm** is a local search algorithm which constantly moves in the direction of (decreasing/increasing) objective value for a given (minimisation/maximisation) problem to find the (lowest/highest) point, also called (nadir/peak), in the search landscape.

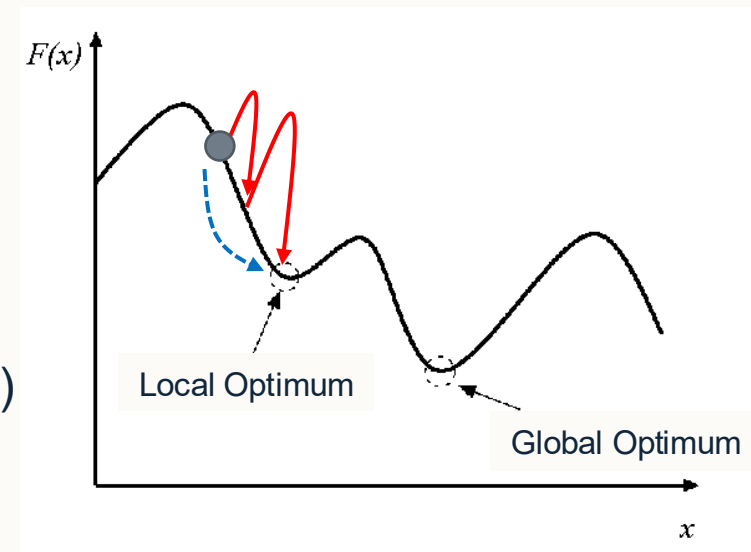


Perturbative Heuristics/Operators

Mutational (diversification/exploration)

VS.

Hill-climbing (intensification/exploitation)





Generic Pseudocode for Hill Climbing

1. **Pick** an initial starting point in the search space.
2. **Repeat**
 1. Consider the neighbour(s) of the current solution as new point(s).
 2. Compare the new point(s) using an evaluation function against the current solution, choosing the best solution among them.
 3. Move to the new point .
3. **Until** a termination criteria is satisfied (number of iterations/until no more improvement possible).
4. **Return** the current point as the best solution found so far.



Notes on Hill Climbing

- Initial starting point can be chosen:
 - Randomly
 - Using a constructive heuristic
 - Based on previous solutions/patterns/etc.
- Hill climbing algorithms differ based on how the new point(s) is/are chosen.



Some Simple Hill Climbing Heuristics

- **Stochastic Hill Climbing** methods randomly sample the neighbourhood of solutions.
 - Random selection/random mutation hill-climbing.
- **Simple Hill Climbing** methods have strategies for systematically sampling the neighbours of a solution.
 - First improvement / next descent / next ascent.
 - Best improvement / steepest descent / steepest ascent.
- **Random Restart** (shotgun) hill climbing builds on other hill-climbing methods by randomly changing the starting solution.



Random Mutation Hill Climbing

```
bestEval = evaluate(currentSolution)
for (k = 0; k < steps; k++) {

    j = random(0, length(currentSolution))
    bitFlip(currentSolution, j) // flip the j^th bit of the solution to produce s' from s
    tempEval = evaluate(currentSolution)

    if(tempEval < bestEval) {
        bestEval = tempEval // accept the bit flip if there is an improvement
    } else {
        bitFlip(currentSolution, j) // otherwise reject the bit flip
    }
}
```



Exercise – Random Mutation Hill Climbing

- $\varphi = (\neg x_0 \vee x_1 \vee x_2) \wedge (x_0 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$
- randomSequence = [2, 0, 2, 3, ...]
- $s = \langle 0, 0, 0, 0 \rangle$ with $f(s) = 1$
- $s' = \langle 0, 0, 1, 0 \rangle$ with $f(s') = 0$ $s \leftarrow s'$ (bestEval = tempEval)
- $s' = \langle 1, 0, 1, 0 \rangle$ with $f(s') = 0$ $s \leftarrow s$ (bitflip(s, j))
- $s' = \langle 0, 0, 0, 0 \rangle \dots$
- $s' = \langle 0, 0, 1, 1 \rangle \dots$
- return $\langle 0, 0, 1, 0 \rangle$

```
bestEval = evaluate(currentSolution)
for (k = 0; k < steps; k++) {

    j = random(0, length(currentSolution))
    bitFlip(currentSolution, j) // flip the j^th bit of the solution to produce s' from s
    tempEval = evaluate(currentSolution)

    if(tempEval < bestEval) {
        bestEval = tempEval // accept the bit flip if there is an improvement
    } else {
        bitFlip(currentSolution, j) // otherwise reject the bit flip
    }
}
```



Note on Improving vs Non-worsening

- [illegible]



First Improvement Hill Climbing

```
bestEval = evaluate(currentSolution)
for (j = 0; j < length(currentSolution); j++) { // performs a single pass of the solution

    bitFlip(currentSolution, j) // flip the j^th bit of the solution to produce s' from s
    tempEval = evaluate(currentSolution)

    if(tempEval < bestEval) {
        bestEval = tempEval // accept the bit flip if there is an improvement
        // optionally we could "break" here
    } else {
        bitFlip(currentSolution, j) // otherwise reject the bit flip
    }
}
```



Example – First Improvement Hill Climbing

- $\varphi = (\neg x_0 \vee x_1 \vee x_2) \wedge (x_0 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$
- $s = \langle 0, 0, 0, 0 \rangle$ with $f(s) = 1$
- $s' = \langle 1, 0, 0, 0 \rangle$ with $f(s') = 1$ $s \leftarrow s$
- $s' = \langle 0, 1, 0, 0 \rangle$ with $f(s') = 1$ $s \leftarrow s$
- $s' = \langle 0, 0, 1, 0 \rangle$ with $f(s') = 0$ $s \leftarrow s'$
- $s' = \langle 0, 0, 1, 1 \rangle$ with $f(s') = 0$ $s \leftarrow s'$

- return ???
- return $\langle 0, 0, 1, 0 \rangle$

```
bestEval = evaluate(currentSolution)
for (j = 0; j < length(currentSolution); j++) { // performs a single pass of the solution

    bitFlip(currentSolution, j) // flip the j^th bit of the solution to produce s' from s
    tempEval = evaluate(solution)

    if(tempEval < bestEval) {
        bestEval = tempEval // accept the bit flip if there is an improvement
        // optionally we could "break" here
    } else {
        bitFlip(currentSolution, j) // otherwise reject the bit flip
    }
}
```



Best Improvement Hill Climbing

```
bestEval = evaluate(currentSolution)
for (j = 0; j < length(currentSolution); j++) { // performs a single pass of the solution

    bitFlip(currentSolution, j) // flip the j^th bit of the solution to produce s' from s
    tempEval = evaluate(currentSolution)

    if(tempEval < bestEval) {
        // remember which bit led to the most improvement
        bestIndex = j
        bestEval = tempEval
        improved = true
    }

    bitFlip(currentSolution, j) // revert the bit flip so we can try another
}

if(improved) bitflip(currentSolution, bestIndex)
```



Example – Best Improvement Hill Climbing

- $\varphi = (\neg x_0 \vee x_1 \vee x_2) \wedge (x_0 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$
- $s = \langle 0, 0, 0, 0 \rangle$ with $f(s) = 1$
- $s' = \langle 1, 0, 0, 0 \rangle$ with $f(s') = 1$ $s \leftarrow s$
- $s' = \langle 0, 1, 0, 0 \rangle$ with $f(s') = 1$ $s \leftarrow s$
- $s' = \langle 0, 0, 1, 0 \rangle$ with $f(s') = 0$ $s \leftarrow s$
- $s' = \langle 0, 0, 0, 1 \rangle$ with $f(s') = 0$ $s \leftarrow s$

- return ???
- return $\langle 0, 0, 1, 0 \rangle$

```
bestEval = evaluate(currentSolution)
for (j = 0; j < length(currentSolution); j++) { // performs a single pass of the solution

    bitFlip(currentSolution, j) // flip the j^th bit of the solution to produce s' from s
    tempEval = evaluate(solution)

    if(tempEval < bestEval) {
        // remember which bit led to the most improvement
        bestIndex = j
        bestEval = tempEval
        improved = true
    }

    bitFlip(currentSolution, j) // revert the bit flip so we can try another
}

if(improved) bitflip(currentSolution, bestIndex)
```




Davis's (Bit) Hill Climbing

```
bestEval = evaluate(currentSolution)
perm = createRandomPermutation(length(currentSolution))
for (j = 0; j < length(currentSolution); j++) { // performs a single pass of the solution

    bitFlip(currentSolution, perm[j]) // flip the bit referenced to in perm's j^th index
    tempEval = evaluate(currentSolution)

    if(tempEval < bestEval) {
        bestEval = tempEval // accept the bit flip
    } else {
        bitFlip(currentSolution, perm[j]) // otherwise reject the bit flip
    }
}
```



Example – Davis's (Bit) Hill Climbing

- $\varphi = (\neg x_0 \vee x_1 \vee x_2) \wedge (x_0 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$
- $\text{perm} = [1, 2, 0, 3]$
- $s = \langle 0, 0, 0, 0 \rangle$ with $f(s) = 1$
- $s' = \langle 0, 1, 0, 0 \rangle$ with $f(s') = 1$ $s \leftarrow s$
- $s' = \langle 0, 0, 1, 0 \rangle$ with $f(s') = 0$ $s \leftarrow s'$
- $s' = \langle 1, 0, 1, 0 \rangle$ with $f(s') = 0$ $s \leftarrow s$
- $s' = \langle 0, 0, 1, 1 \rangle$ with $f(s') = 0$ $s \leftarrow s$

- **return** $\langle 0, 0, 1, 0 \rangle$

```
bestEval = evaluate(currentSolution)
perm = createRandomPermutation(length(currentSolution))
for (j = 0; j < length(currentSolution); j++) { // performs a single pass of the solution

    bitFlip(currentSolution, perm[j]) // flip the bit referenced to in perm's j^th index
    tempEval = evaluate(currentSolution)

    if(tempEval < bestEval) {
        bestEval = tempEval // accept the bit flip
    } else {
        bitFlip(currentSolution, j) // otherwise reject the bit flip
    }
}
```



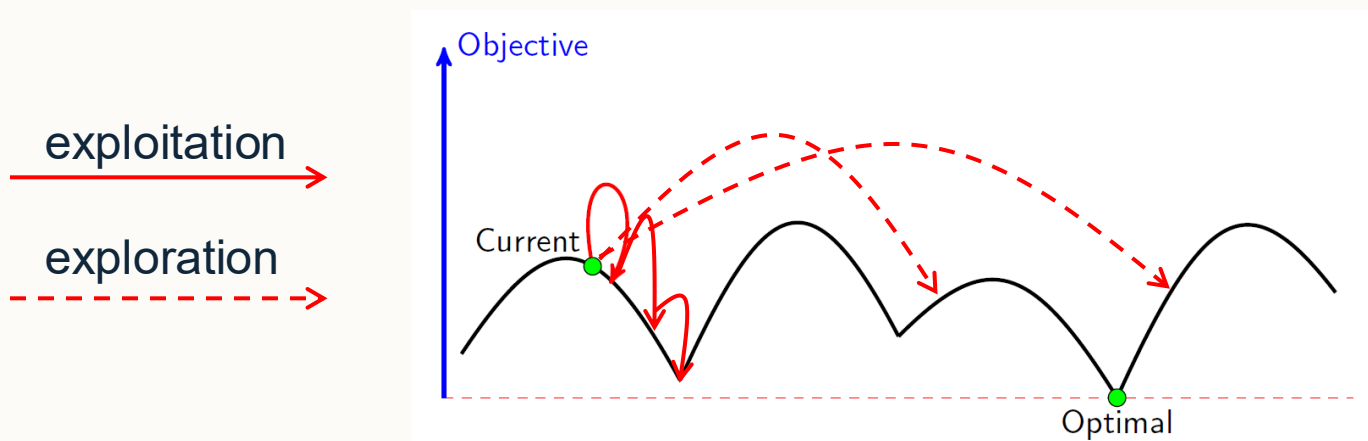
When to Stop Hill Climbing

- If the target objective value is known, we can stop if this is achieved.
 - E.g. MAX-SAT 0 for minimising or n the number of clauses for maximising. (but what if the instance is UNSAT?).
- Hill climbing algorithms can be applied repeatedly until some termination criterion is met, however:
 - No point in applying best improvement/next improvement/DBHC if there is no improvement in the previous pass.
- **Home task** – convince yourself it is true that there is no point keep applying the three hill climbing methods as soon as no improvement is made in the previous pass.



Hill Climbing vs Random Walk

- **Hill-climbing** methods exploit the best available solution but avoids exploring a large portion of the search space.
- **Random walk** explores the whole search space but does not exploit any promising regions.
- Art of designing a good (meta)heuristic algorithm





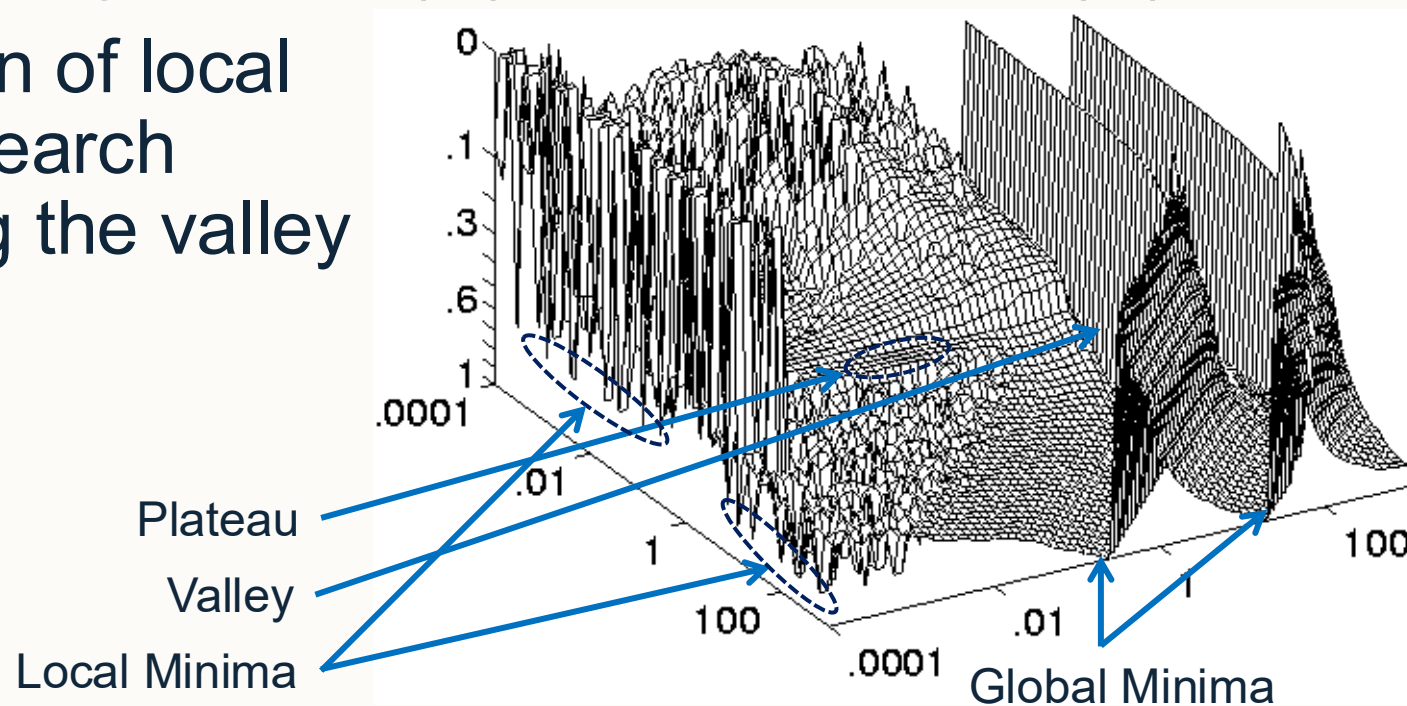
Strengths of Hill Climbing Methods

- Very easy to implement requiring only:
 - A solution representation,
 - An evaluation function, and
 - A way to define the neighbourhood of a solution.



Weaknesses of Hill Climbing Methods 1

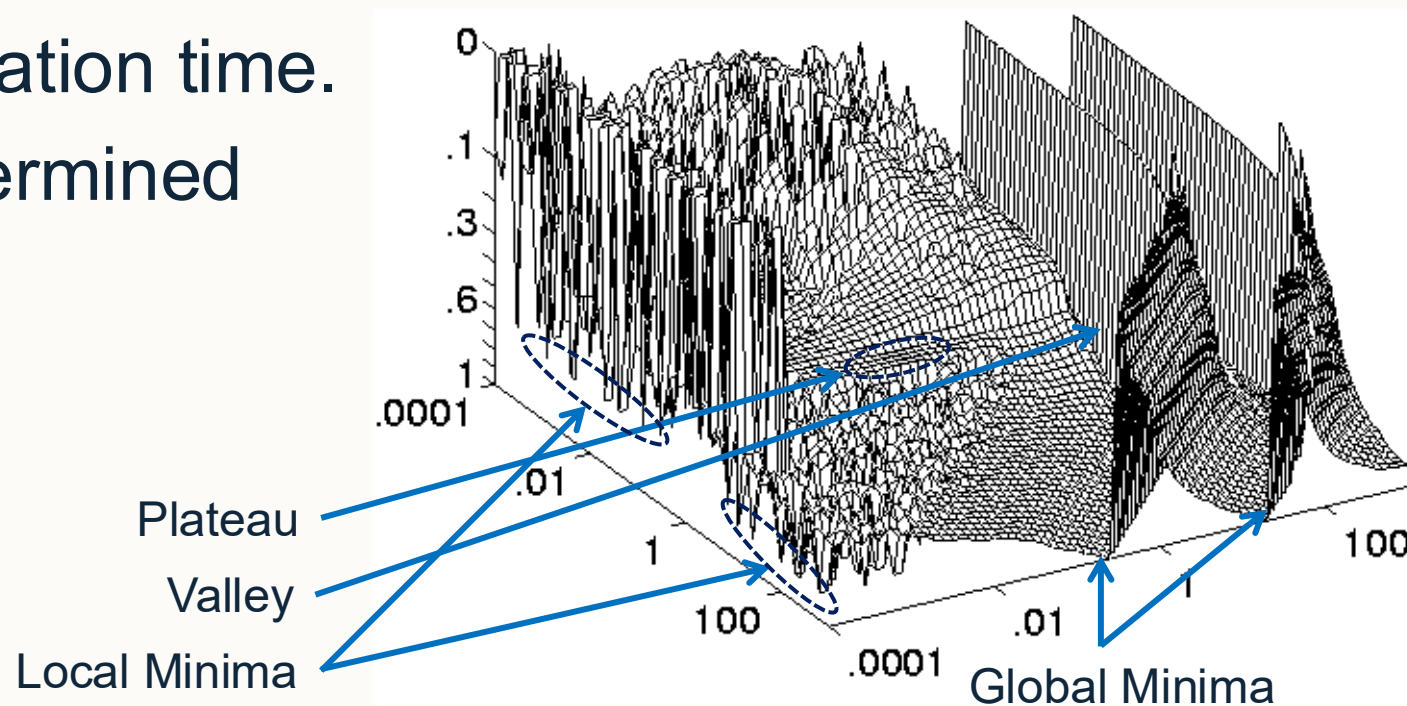
- **Local optimum:** hill-climbing will get stuck as all neighbouring solutions have a worse (or same) quality.
- **Plateau/neutral space/shoulder:** all neighbouring states have the same quality; hill-climbing will get stuck ($<$) or random walk ($\leq$).
- **Ridge/valley:** a combination of local optimum and plateau; the search explores the solutions along the valley but is unable to escape.





Weaknesses of Hill Climbing Methods 2

- No guarantees hill climbing will find the global optimum as can easily get stuck in the aforementioned search space features.
- No information to know how much a given local optimum deviates from the global optimum.
- No upper bound on computation time.
- Success/failure usually determined based on the starting point.





Concluding remarks

- Take-home exercise (see Moodle):
 - Design of components for solving Examination Timetabling – discuss your solutions on the forum.
- Module activities:
 - Formative self-assessment quiz on Moodle.
 - Computing Session (COMP2001): Friday 6th February CS-A32 4pm – Hill-Climbing.
 - Next lecture:
 - Warren will take over for 2-weeks.
 - 10th February 4pm – Metaheuristics.



University of
Nottingham

UK | CHINA | MALAYSIA

Thank you

Any questions?